HW0

Ziyu Ge

(a)

Let

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

denote the rotation matrix by angle θ. Reflection about the x-axis is given by

$$J = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Reflection about the line through the origin making angle θ with the x-axis can be expressed as a conjugation of J by a rotation:

$$F_\theta = R(\theta)\, J\, R(-\theta).$$

Consider reflections about the lines $\theta = \alpha$ and $\theta = \beta$. Their composition is

$$F_\beta F_\alpha = \big(R(\beta)JR(-\beta)\big)\big(R(\alpha)JR(-\alpha)\big).$$

Using the identity $R(-\beta)R(\alpha) = R(\alpha - \beta)$,

$$F_\beta F_\alpha = R(\beta)\, J\, R(\alpha - \beta)\, J\, R(-\alpha).$$

For any angle γ, compute

$$JR(\gamma)J = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} \cos\gamma & -\sin\gamma \\ \sin\gamma & \cos\gamma \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} \cos\gamma & \sin\gamma \\ -\sin\gamma & \cos\gamma \end{pmatrix} = R(-\gamma).$$

Applying this identity with $\gamma = \alpha - \beta$,

$$JR(\alpha - \beta)J = R(\beta - \alpha).$$

$$F_\beta F_\alpha = R(\beta)\, R(\beta - \alpha)\, R(-\alpha).$$

Since multiplication of rotation matrices corresponds to addition of angles,

$$R(\beta)\, R(\beta - \alpha)\, R(-\alpha) = R\big(\beta + (\beta - \alpha) - \alpha\big) = R\big(2(\beta - \alpha)\big).$$

Thus, reflection about the line $\theta = \alpha$ followed by reflection about the line $\theta = \beta$ is equivalent to a rotation of angle

$$2(\beta - \alpha).$$

(b)

Let $s \in \mathbb{R}^3$ be a unit vector ($\|s\| = 1$). Define the skew-symmetric matrix $\hat{s}$ by the property

$$\hat{s}x = s \times x \qquad \text{for all } x \in \mathbb{R}^3,$$

i.e.

$$\hat{s} = \begin{pmatrix} 0 & -s_3 & s_2 \\ s_3 & 0 & -s_1 \\ -s_2 & s_1 & 0 \end{pmatrix}.$$

Consider the matrix exponential

$$R(\phi) = e^{\phi\hat{s}} = \sum_{k=0}^{\infty} \frac{(\phi\hat{s})^k}{k!} = I + \phi\hat{s} + \frac{\phi^2}{2!}\hat{s}^2 + \frac{\phi^3}{3!}\hat{s}^3 + \cdots .$$

For any $x \in \mathbb{R}^3$,

$$\hat{s}^2 x = \hat{s}(\hat{s}x) = s \times (s \times x).$$

Using the vector triple product identity

$$a \times (b \times c) = b(a \cdot c) - c(a \cdot b),$$

with $a = b = s$ and $c = x$,

$$s \times (s \times x) = s(s \cdot x) - x(s \cdot s).$$

Since $\|s\| = 1$, $s \cdot s = 1$, hence

$$\hat{s}^2 x = s(s \cdot x) - x.$$

Equivalently, as a matrix identity,

$$\hat{s}^2 = ss^{\mathsf{T}} - I.$$

Next,

$$\hat{s}^3 x = \hat{s}(\hat{s}^2 x) = \hat{s}\big(s(s \cdot x) - x\big) = \hat{s}\big(s(s \cdot x)\big) - \hat{s}x.$$

But $\hat{s}s = s \times s = 0$, so $\hat{s}\big(s(s \cdot x)\big) = (s \cdot x)\hat{s}s = 0$. Therefore,

$$\hat{s}^3 x = -\hat{s}x \quad \Rightarrow \quad \hat{s}^3 = -\hat{s}.$$

Multiplying by $\hat{s}^2$ repeatedly also gives the pattern

$$\hat{s}^{2k+1} = (-1)^k \hat{s}, \qquad \hat{s}^{2k+2} = (-1)^k \hat{s}^2 \qquad (k \geq 0).$$

Using the power pattern above,

$$e^{\phi\hat{s}} = I + \sum_{k=0}^{\infty} \frac{\phi^{2k+1}}{(2k+1)!}\hat{s}^{2k+1} + \sum_{k=0}^{\infty} \frac{\phi^{2k+2}}{(2k+2)!}\hat{s}^{2k+2}$$
$$= I + \sum_{k=0}^{\infty} \frac{\phi^{2k+1}}{(2k+1)!}(-1)^k\hat{s} + \sum_{k=0}^{\infty} \frac{\phi^{2k+2}}{(2k+2)!}(-1)^k\hat{s}^2.$$

Factor out $\hat{s}$ and $\hat{s}^2$:

$$e^{\phi\hat{s}} = I + \left(\sum_{k=0}^{\infty} \frac{(-1)^k\phi^{2k+1}}{(2k+1)!}\right)\hat{s} + \left(\sum_{k=0}^{\infty} \frac{(-1)^k\phi^{2k+2}}{(2k+2)!}\right)\hat{s}^2.$$

Recognize the scalar Taylor series:

$$\sum_{k=0}^{\infty} \frac{(-1)^k\phi^{2k+1}}{(2k+1)!} = \sin\phi, \qquad \sum_{k=0}^{\infty} \frac{(-1)^k\phi^{2k+2}}{(2k+2)!} = 1 - \cos\phi.$$

Therefore,

$$e^{\phi\hat{s}} = I + (\sin\phi)\hat{s} + (1 - \cos\phi)\hat{s}^2.$$

Thus, Rodrigues' rotation formula is verified:

$$\boxed{R(\phi) = e^{\phi\hat{s}} = I + \sin\phi\,\hat{s} + (1 - \cos\phi)\,\hat{s}^{\,2}}.$$

(c) Code and outputs are attached as Appendix 1

I implemented Rodrigues' formula

$$R = I + \sin\phi\,\hat{u} + (1-\cos\phi)\hat{u}^2, \qquad u = \frac{s}{\|s\|},$$

where $\hat{u}$ is the skew-symmetric matrix such that $\hat{u}x = u \times x$.

For the example input $s = (1,2,3)^{\mathsf{T}}$ and $\phi = 0.7$, the code produced

$$R = \begin{pmatrix} 0.78163917 & -0.48292928 & 0.39473980 \\ 0.55011723 & 0.83203013 & -0.07139250 \\ -0.29395788 & 0.27295634 & 0.91601507 \end{pmatrix}.$$

The eigenvalues returned by `numpy.linalg.eig` are

$$\lambda_1 \approx 0.76484219 + 0.64421769\,i, \qquad \lambda_2 \approx 0.76484219 - 0.64421769\,i, \qquad \lambda_3 \approx 1.$$

Note that

$$0.76484219 \approx \cos(0.7), \qquad 0.64421769 \approx \sin(0.7),$$

so

$$\lambda_1 \approx e^{i\phi}, \qquad \lambda_2 \approx e^{-i\phi}, \qquad \lambda_3 = 1,$$

which matches the standard spectrum of a 3D rotation: the rotation fixes the axis direction and rotates the orthogonal plane by angle ϕ.

To verify that the axis direction is an eigenvector with eigenvalue 1, let

$$u = \frac{s}{\|s\|}.$$

The code computed

$$\|Ru - u\| = 1.5700924586837752 \times 10^{-16},$$

which is essentially 0 up to floating-point error. Hence $Ru = u$, confirming that the rotation leaves the axis direction invariant.

The code outputs

$$\cos(\phi) = 0.7648421872844885, \qquad \frac{1}{2}\big(\mathrm{trace}(R) - 1\big) = 0.7648421872844884,$$

and the absolute difference

$$\big|\cos(\phi) - \tfrac{1}{2}(\mathrm{trace}(R) - 1)\big| = 1.1102230246251565 \times 10^{-16}.$$

Thus, the numerical results verify the identity

$$\cos\phi = \frac{1}{2}\big(\operatorname{trace}(R) - 1\big).$$

For each point x, the rotated point is $x' = Rx$. The code reported:

$$\begin{aligned}
(1,0,0)^{\mathsf{T}} &\mapsto (0.78163917,\ \ 0.55011723,\ -0.29395788)^{\mathsf{T}},\\
(0,1,0)^{\mathsf{T}} &\mapsto (-0.48292928,\ \ 0.83203013,\ \ 0.27295634)^{\mathsf{T}},\\
(0,0,1)^{\mathsf{T}} &\mapsto (0.39473980,\ -0.07139250,\ \ 0.91601507)^{\mathsf{T}},\\
(1,1,1)^{\mathsf{T}} &\mapsto (0.69344969,\ \ 1.31075487,\ \ 0.89501353)^{\mathsf{T}}.
\end{aligned}$$

These examples illustrate the action of the rotation matrix on several points in $\mathbb{R}^3$.

(d) Code and outputs are attached as Appendix 1

Using the identities

$$\cos\phi = \frac{1}{2}\big(\mathrm{trace}(R) - 1\big), \qquad R - R^{\mathsf{T}} = 2\sin\phi\,\hat{s},$$

I implemented a Python function that recovers the rotation angle and axis from a given rotation matrix.

For the rotation matrix R obtained in part (c), the code outputs

$$s = \begin{pmatrix} 0.26726124 \\ 0.53452249 \\ 0.80178372 \end{pmatrix}, \qquad \phi = 0.7000000035461436.$$

The recovered axis s is a unit vector and satisfies

$$s \approx \frac{1}{\sqrt{14}}(1, 2, 3)^{\mathsf{T}},$$

which agrees with the original axis direction used to generate R in part (c), up to normalization. This confirms that the rotation leaves the axis direction invariant.

From the trace of R, the code computes

$$\cos\phi = 0.764842185, \qquad \frac{1}{2}\big(\mathrm{trace}(R) - 1\big) = 0.764842185.$$

The two values coincide up to floating-point precision, verifying the identity

$$\cos\phi = \frac{1}{2}\big(\mathrm{trace}(R) - 1\big).$$

Thus, the numerical results confirm that the implemented inverse mapping correctly recovers both the rotation axis s and the rotation angle ϕ from the orthogonal matrix R.

(e) Let the centroids be

$$\bar{u} = \frac{1}{4}\sum_{j=1}^{4} u_j, \qquad \bar{v} = \frac{1}{4}\sum_{j=1}^{4} v_j.$$

Compute:

$$\bar{u} = \frac{1}{4}\big((-3,0) + (1,1) + (1,0) + (1,-1)\big) = (0,0),$$

$$\bar{v} = \frac{1}{4}\big((0,3) + (1,0) + (0,0) + (-1,0)\big) = \left(0, \frac{3}{4}\right).$$

Define centered points

$$\tilde{u}_j = u_j - \bar{u} = u_j, \qquad \tilde{v}_j = v_j - \bar{v}.$$

Then minimizing $\sum_j \|Ru_j + t - v_j\|^2$ over (R,t) with $R \in SO(2)$ is equivalent to:

$$t^\star = \bar{v} - R\bar{u} = \bar{v},$$

and

$$R^\star = \arg\min_{R \in SO(2)} \sum_{j=1}^{4} \|R\tilde{u}_j - \tilde{v}_j\|^2.$$

So the optimal translation is immediately

$$\boxed{t^\star = \left(0, \frac{3}{4}\right)^{\mathsf{T}}}.$$

Let

$$U = \begin{pmatrix} \tilde{u}_1 & \tilde{u}_2 & \tilde{u}_3 & \tilde{u}_4 \end{pmatrix}, \qquad V = \begin{pmatrix} \tilde{v}_1 & \tilde{v}_2 & \tilde{v}_3 & \tilde{v}_4 \end{pmatrix}.$$

The standard Procrustes solution uses the 2×2 cross-covariance matrix

$$H = UV^{\mathsf{T}} = \sum_{j=1}^{4} \tilde{u}_j \tilde{v}_j^{\mathsf{T}}.$$

Compute $\tilde{v}_j = v_j - \bar{v}$:

$$\tilde{v}_1 = \left(0, \frac{9}{4}\right),\ \tilde{v}_2 = \left(1, -\frac{3}{4}\right),\ \tilde{v}_3 = \left(0, -\frac{3}{4}\right),\ \tilde{v}_4 = \left(-1, -\frac{3}{4}\right).$$

Then

$$H = \sum_{j=1}^{4} \tilde{u}_j \tilde{v}_j^{\mathsf{T}} = \begin{pmatrix} 0 & -9 \\ 2 & 0 \end{pmatrix}.$$

Take the SVD:

$$H = U_H \Sigma V_H^{\mathsf{T}},$$

and the minimizing rotation is

$$R^\star = V_H U_H^\mathsf{T} \quad (\text{with } \det(R^\star) = 1).$$

Carrying this out gives

$$\boxed{R^\star = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}}$$

which is a rotation by $-\frac{\pi}{2}$ (clockwise 90°).

Therefore, the least-squares Euclidean transformation is

$$\boxed{E(u) = R^\star u + t^\star, \qquad R^\star = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad t^\star = \begin{pmatrix} 0 \\ \frac{3}{4} \end{pmatrix}.}$$

(f) Show that the vanishing points of lines on a plane lie on the vanishing line of the plane.

Use the standard pinhole camera model in homogeneous coordinates. Let the camera matrix be $P = K\,[R \mid t]$, and let a 3D point $X \in \mathbb{P}^3$ project to an image point $x \sim PX \in \mathbb{P}^2$.

Let π be a 3D plane with homogeneous equation

$$\pi^{\mathsf{T}} X = 0.$$

The image of the plane π is a planar region in the image, and the vanishing line of the plane is defined as the image of the line at infinity on that plane. Equivalently, it is the image of the set of points X_∞ that lie on both the plane and the plane at infinity:

$$\pi_\infty^{\mathsf{T}} X = 0, \qquad \pi_\infty = (0,0,0,1)^{\mathsf{T}}.$$

Thus the line at infinity on π is

$$L_\infty(\pi) \;=\; \{\, X \in \mathbb{P}^3 :\; \pi^{\mathsf{T}} X = 0,\; \pi_\infty^{\mathsf{T}} X = 0 \,\}.$$

The vanishing line of π is

$$\ell_\pi \;=\; \{\, x \in \mathbb{P}^2 :\; x \sim PX,\; X \in L_\infty(\pi) \,\}.$$

Let L be any 3D line lying on the plane π. Since $L \subset \pi$, every point on L satisfies $\pi^{\mathsf{T}} X = 0$.

In projective 3D geometry, any line L meets the plane at infinity π_∞ in exactly one point (unless $L \subset \pi_\infty$), called the *point at infinity* of the line. Denote this intersection by

$$X_v \;=\; L \cap \pi_\infty.$$

Because $L \subset \pi$, we also have $X_v \in \pi$; therefore

$$X_v \in \pi \cap \pi_\infty = L_\infty(\pi).$$

The *vanishing point* of the line L is the image of this point at infinity:

$$x_v \sim PX_v.$$

Since $X_v \in L_\infty(\pi)$ and ℓ_π is precisely the image of $L_\infty(\pi)$ under the projection P, it follows that

$$x_v \in \ell_\pi.$$

Because the choice of the line $L \subset \pi$ was arbitrary, this proves that *the vanishing points of all lines on the plane π lie on the vanishing line of the plane.* □

(a) Are we strictly required to use our atomic operations when defining new functions (e.g., sigmoid)? Under what conditions can we define new operations?

We are not required to define new functions only using atomic operations. A new operation can be defined directly as long as its forward computation and its backward gradient are correctly implemented. As long as the chain rule is applied correctly, functions such as tanh, exp, or sigmoid can be implemented without decomposing them into atomic operations.

(b) When performing backpropagation on a Value, why do we accumulate the gradient instead of directly assigning it?

We accumulate gradients because a Value can contribute to the output through multiple paths in the computational graph. By the chain rule, the total gradient is the sum of gradients from all downstream paths. Therefore, gradients must be added rather than overwritten to correctly account for all contributions.

Appendix 1

(c)

```
import numpy as np

def rodrigues_R(s, phi):
    """Rotation matrix for angle phi about axis vector s (3,)."""
    s = np.asarray(s, dtype=float)
    s = s / np.linalg.norm(s)  # normalize axis
    sx, sy, sz = s
    K = np.array([[0,   -sz,  sy],
                  [sz,   0,  -sx],
                  [-sy, sx,   0]], dtype=float)
    I = np.eye(3)
    return I + np.sin(phi) * K + (1 - np.cos(phi)) * (K @ K)

# example: axis, angle
s = np.array([1.0, 2.0, 3.0])
phi = 0.7

R = rodrigues_R(s, phi)
print("R=\n", R)

# eigenvalues / eigenvectors
w, V = np.linalg.eig(R)
print("\neigenvalues:\n", w)

# axis direction should be eigenvector with eigenvalue about 1
u = s / np.linalg.norm(s)
axis_err = np.linalg.norm(R @ u - u)
print("\n||Ru - u|| =", axis_err)
```

```python
# trace identity check
lhs = np.cos(phi)
rhs = 0.5 * (np.trace(R) - 1)
print("\ncos(phi) =", lhs)
print("0.5*(trace(R)-1) =", rhs)
print("abs diff =", abs(lhs - rhs))

# show points before/after
points = np.array([[1,0,0],
                   [0,1,0],
                   [0,0,1],
                   [1,1,1]], dtype=float)

rot = (R @ points.T).T
print("\npoints: x -> Rx")
for x, y in zip(points, rot):
    print(x, "->", y)
```

```
R=
 [[ 0.78163917 -0.48292928  0.3947398 ]
 [ 0.55011723  0.83203013 -0.0713925 ]
 [-0.29395788  0.27295634  0.91601507]]

eigenvalues:
 [0.76484219+0.64421769j 0.76484219-0.64421769j 1.        +0.j        ]

||Ru - u|| = 1.5700924586837752e-16

cos(phi) = 0.7648421872844885
0.5*(trace(R)-1) = 0.7648421872844884
abs diff = 1.1102230246251565e-16

points: x -> Rx
[1. 0. 0.] -> [ 0.78163917  0.55011723 -0.29395788]
[0. 1. 0.] -> [-0.48292928  0.83203013  0.27295634]
[0. 0. 1.] -> [ 0.3947398  -0.0713925   0.91601507]
[1. 1. 1.] -> [0.69344969 1.31075487 0.89501353]
```

(d)

```
import numpy as np

def axis_angle_from_R(R, eps=1e-12):
    """
    Given a 3x3 rotation matrix R (orthogonal, det ~ 1),
    return (s, phi) where s is the unit axis and phi is the rotation angle in [0, pi].

    Uses:
      cos(phi) = (trace(R) - 1)/2
      R - R^T = 2 sin(phi) * hat(s)
    Handles near-0 and near-pi angles with simple stable fallbacks.
    """
    R = np.asarray(R, dtype=float)

    # angle from trace
    c = (np.trace(R) - 1.0) / 2.0
    c = np.clip(c, -1.0, 1.0)
    phi = np.arccos(c)

    # near 0: axis is not unique; return a default axis
    if phi < 1e-8:
        return np.array([1.0, 0.0, 0.0]), 0.0

    # general case: use skew part to get axis
    if abs(np.sin(phi)) > 1e-8:
        A = (R - R.T) / (2.0 * np.sin(phi))  # should equal hat(s)
        s = np.array([A[2, 1], A[0, 2], A[1, 0]])
        s = s / np.linalg.norm(s)
        return s, phi

    # near pi: sin(phi) ~ 0, use diagonal-based recovery
    # For phi ~ pi, R   2 s s^T - I   =>   (R + I)/2   s s^T
    M = (R + np.eye(3)) / 2.0
    s = np.array([np.sqrt(max(M[0, 0], 0.0)),
                  np.sqrt(max(M[1, 1], 0.0)),
                  np.sqrt(max(M[2, 2], 0.0))])

    # choose signs using off-diagonals (if possible)
    if M[0, 1] < 0: s[1] = -s[1]
```

```
    if M[0, 2] < 0: s[2] = -s[2]
    # normalize (in case of small numerical drift)
    if np.linalg.norm(s) < eps:
        # fallback if degenerate
        s = np.array([1.0, 0.0, 0.0])
    else:
        s = s / np.linalg.norm(s)
    return s, phi


# quick test using your previous R (optional)
if __name__ == "__main__":
    R = np.array([[ 0.78163917, -0.48292928,  0.3947398 ],
                  [ 0.55011723,  0.83203013, -0.0713925 ],
                  [-0.29395788,  0.27295634,  0.91601507]])
    s, phi = axis_angle_from_R(R)
    print("axis s =", s)
    print("phi =", phi)
    print("cos(phi) from trace =", 0.5*(np.trace(R)-1))
    print("cos(phi) =", np.cos(phi))
```

```
axis s = [0.26726124 0.53452249 0.80178372]
phi = 0.7000000035461436
cos(phi) from trace = 0.764842185
cos(phi) = 0.764842185
```

```
import math
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

```
from graphviz import Digraph

def trace(root):
  # builds a set of all nodes and edges in a graph
  nodes, edges = set(), set()
  def build(v):
    if v not in nodes:
      nodes.add(v)
      for child in v._prev:
        edges.add((child, v))
        build(child)
  build(root)
  return nodes, edges

def draw_dot(root):
  dot = Digraph(format='svg', graph_attr={'rankdir': 'LR'}) # LR = left to right

  nodes, edges = trace(root)
  for n in nodes:
    uid = str(id(n))
    # for any value in the graph, create a rectangular ('record') node for it
    dot.node(name = uid, label = "{ %s | data %.4f | grad %.4f }" % (n.label, n.data, n.grad
    if n._op:
      # if this value is a result of some operation, create an op node for it
      dot.node(name = uid + n._op, label = n._op)
      # and connect this node to it
      dot.edge(uid + n._op, uid)

  for n1, n2 in edges:
    # connect n1 to the op node of n2
    dot.edge(str(id(n1)), str(id(n2)) + n2._op)

  return dot
```

```
class Value:
    def __init__(self, data, _children=(), _op='', label=''):
        self.label = label
```

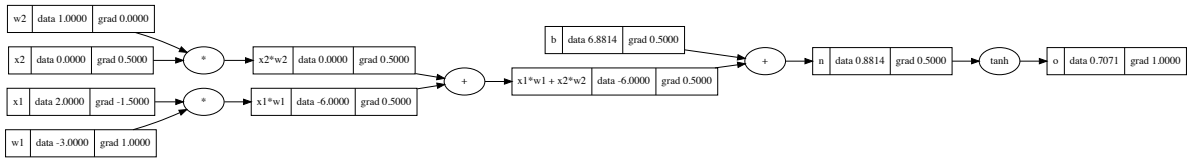
```
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __repr__(self):
        return f"Value(data={self.data})"

    def __rmul__(self, other):
        return self * other

    def __radd__(self, other):
        return self + other

    def __sub__(self, other):
        return self + (-other)

    def __neg__(self):
        return self * -1

    def __add__(self, other):

        other = other if isinstance(other, Value) else Value(other)

        out = Value(self.data + other.data, (self, other), '+')

        def _backward():
            # TODO:1
            self.grad += out.grad
            other.grad += out.grad
        out._backward = _backward

        return out

    def __mul__(self, other):

        other = other if isinstance(other, Value) else Value(other)

        out = Value(self.data * other.data, (self, other), '*')

        def _backward():
```

```
            # TODO:2
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward

        return out

    def tanh(self):
        x = self.data
        # TODO:3a
        t = math.tanh(x)
        out = Value(t, (self,), 'tanh')

        def _backward():
            # TODO:3b
            self.grad += (1 - out.data**2) * out.grad
        out._backward = _backward

        return out

    def exp(self):
        x = self.data
        # TODO:4a
        t = math.exp(x)
        out = Value(t, (self,), 'exp')

        def _backward():
            # TODO:4b
            self.grad += out.data * out.grad
        out._backward = _backward

        return out

    def __truediv__(self, other):
        return self * other**-1

    def __pow__(self, other):
        assert isinstance(other, (int, float))
        out = Value(self.data ** other, (self, ), f'pow({other})')

        def _backward():
            # TODO:5
```

```
            self.grad += other * (self.data ** (other - 1)) * out.grad
        out._backward = _backward

        return out

    def backward(self):

        list_ = []
        visited = set()

        def build_topo(node):
          if node not in visited:
            visited.add(node)
            for child in node._prev:
              build_topo(child)
            list_.append(node)

        build_topo(self)

        self.grad = 1.0
        for node in reversed(list_):
          node._backward()
```

```
# inputs x1,x2
x1 = Value(2.0, label='x1')
x2 = Value(0.0, label='x2')
# weights w1,w2
w1 = Value(-3.0, label='w1')
w2 = Value(1.0, label='w2')
# bias of the neuron
b = Value(6.8813735870195432, label='b')
# x1*w1 + x2*w2 + b
x1w1 = x1*w1; x1w1.label = 'x1*w1'
x2w2 = x2*w2; x2w2.label = 'x2*w2'
x1w1x2w2 = x1w1 + x2w2; x1w1x2w2.label = 'x1*w1 + x2*w2'
n = x1w1x2w2 + b; n.label = 'n'
o = n.tanh(); o.label = 'o'
```

```
# visualize the gradients and forward
o.backward()
draw_dot(o)
```




```
# inputs x1,x2
x1 = Value(2.0, label='x1')
x2 = Value(0.0, label='x2')
# weights w1,w2
w1 = Value(-3.0, label='w1')
w2 = Value(1.0, label='w2')
# bias of the neuron
b = Value(6.8813735870195432, label='b')
# x1*w1 + x2*w2 + b
x1w1 = x1*w1; x1w1.label = 'x1*w1'
x2w2 = x2*w2; x2w2.label = 'x2*w2'
x1w1x2w2 = x1w1 + x2w2; x1w1x2w2.label = 'x1*w1 + x2*w2'
n = x1w1x2w2 + b; n.label = 'n'
o1 = 2*n; o1.label = 'o1'
o2 = o1.exp(); o2.label = 'o2'
o = (o2-1) / (o2+1); o.label = 'o'
```

```
# visualize the gradients and forward
o.backward()
draw_dot(o)
```

```
import random
class Neuron:

  def __init__(self, nin):
    self.w = [Value(random.uniform(-1,1)) for _ in range(nin)]
    self.b = Value(random.uniform(-1,1))

  def __call__(self, x):
    # w * x + b
    act = sum((wi*xi for wi, xi in zip(self.w, x)), self.b)
    out = act.tanh()
    return out
```

```
  def parameters(self):
    return self.w + [self.b]

class Layer:

  def __init__(self, nin, nout):
    self.neurons = [Neuron(nin) for _ in range(nout)]

  def __call__(self, x):
    outs = [n(x) for n in self.neurons]
    return outs[0] if len(outs) == 1 else outs

  def parameters(self):
    return [p for neuron in self.neurons for p in neuron.parameters()]

class MLP:

  def __init__(self, nin, nouts):
    sz = [nin] + nouts
    self.layers = [Layer(sz[i], sz[i+1]) for i in range(len(nouts))]

  def __call__(self, x):
    for layer in self.layers:
      x = layer(x)
    return x

  def parameters(self):
    return [p for layer in self.layers for p in layer.parameters()]
```

```
xs = [
  [2.0, 3.0, -1.0],
  [3.0, -1.0, 0.5],
  [0.5, 1.0, 1.0],
  [1.0, 1.0, -1.0],
]
ys = [1.0, -1.0, -1.0, 1.0] # desired targets
```

```
# define a mlp and train a model
mlp = MLP(3, [4, 4, 1])
```

```
for e in range(10):
  params = mlp.parameters()

  pred = [mlp(x) for x in xs]
  loss = sum([(y_pred - y)**2 for y_pred, y in zip(pred, ys)])
  for p_ in params:
    p_.grad = 0

  loss.backward()

  for p_ in params:
    p_.data += -0.05 * p_.grad

  print(loss)
```

```
Value(data=1.8339137487593313)
Value(data=0.41555699029602783)
Value(data=0.28197811284279284)
Value(data=0.21401406264562522)
Value(data=0.1716531975826992)
Value(data=0.14264103384203744)
Value(data=0.12155911719364693)
Value(data=0.10558450625545035)
Value(data=0.09309093901656358)
Value(data=0.08307337827903191)
```

```
print([mlp(x) for x in xs])
```

```
[Value(data=0.8899456458140935), Value(data=-0.8655791089754525), Value(data=-
0.8667031852635355), Value(data=0.835902096343844)]
```